

MIS 6326: Open Test - R

Name: Mugizi Polycarp

Registration Number: 2024-M132-24001

4. Customer Churn Prediction

Dataset: [Telco Churn \(Kaggle\)](#)

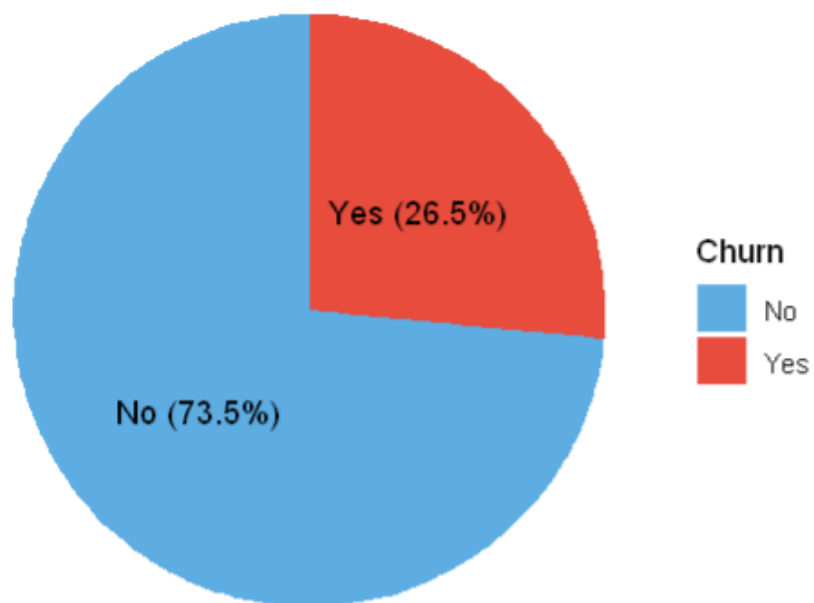
1. First EDA (Raw Data)

- Summary stats of tenure, charges.

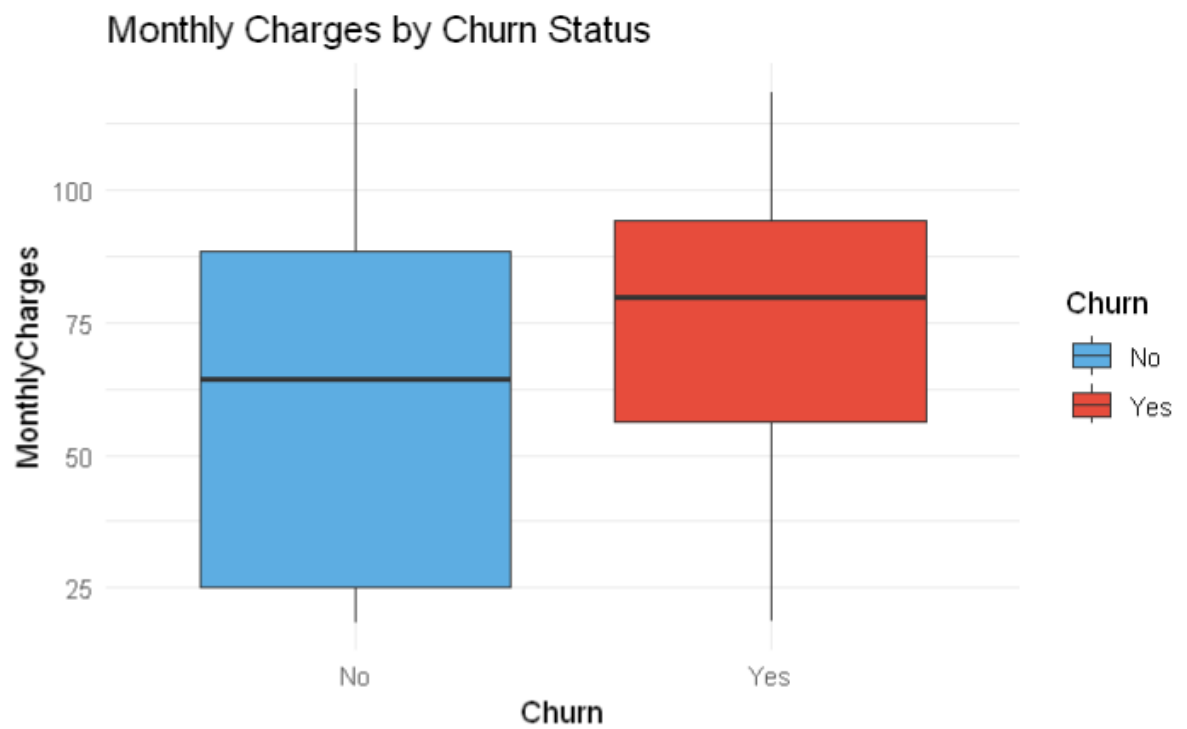
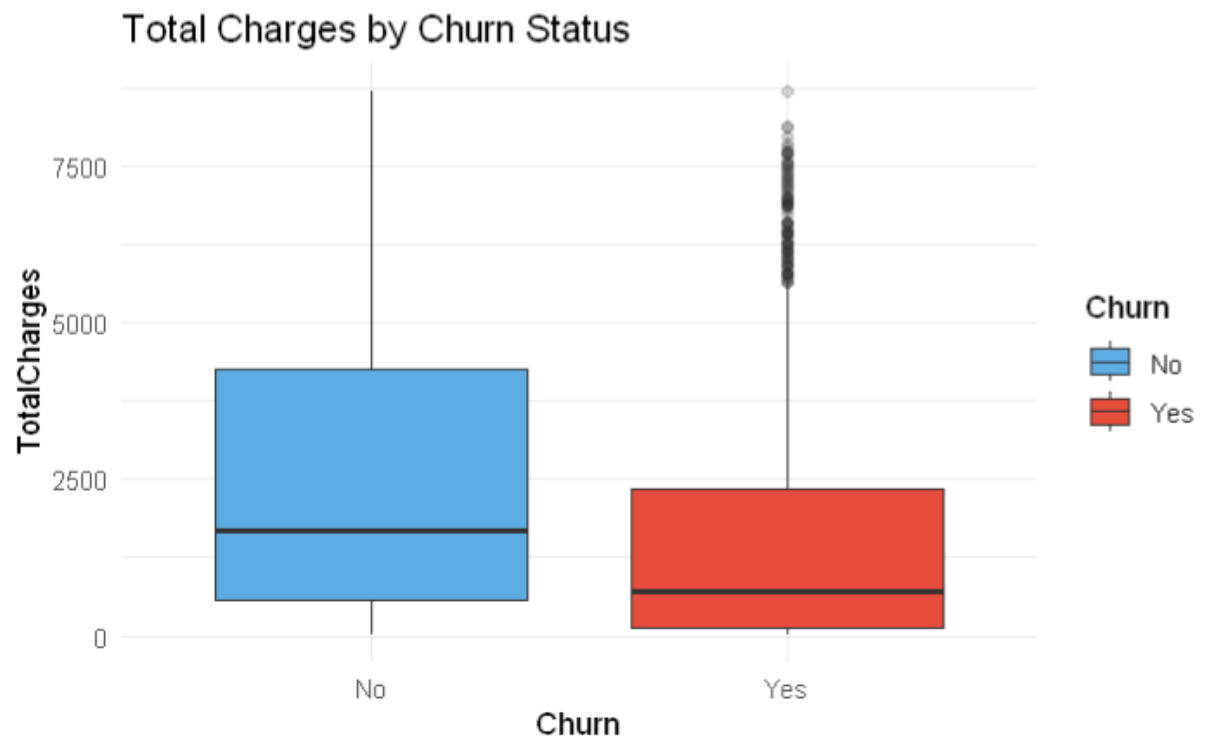
	tenure_min	tenure_max	tenure_mean	tenure_median	monthly_min	monthly_max	monthly_mean	monthly_median	total_min	total_max	total_mean	total_median
1	0	72	32.37115	29	18.25	118.75	64.76169	70.35	18.8	8684.8	2283.3	1397.475

- Churn distribution (bar chart).

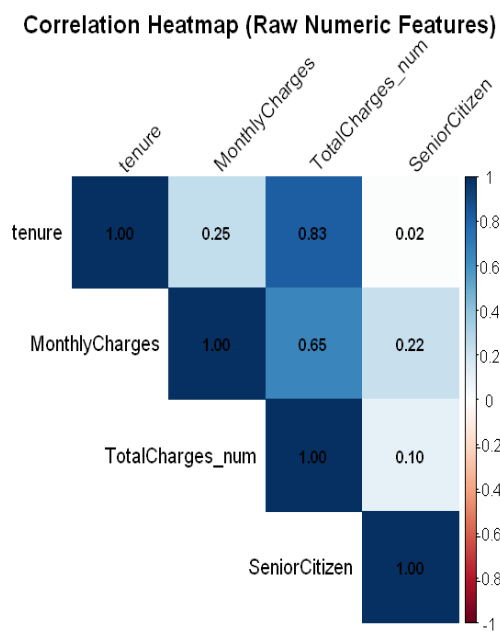
Churn Distribution (Pie Chart)



- Boxplots: churn vs non-churn for charges.



- **Correlation heatmap for numerical features.**



R code

```

1
2 # EDA on Raw Data
3 # Load required libraries
4 library(ggplot2)
5 library(dplyr)
6 library(corrplot)
7 library(reshape2)
8
9 #Load raw data
10 data_path <- "D:\\Msc\\datasets\\3\\Churn.csv"
11 telco <- read.csv(data_path, stringsAsFactors = FALSE)
12
13 # Summary statistics of tenure and charges
14 summary_stats <- telco %>%
15   summarise(
16     tenure_min = min(tenure, na.rm = TRUE),
17     tenure_max = max(tenure, na.rm = TRUE),
18     tenure_mean = mean(tenure, na.rm = TRUE),
19     tenure_median = median(tenure, na.rm = TRUE),
20
21     monthly_min = min(MonthlyCharges, na.rm = TRUE),
22     monthly_max = max(MonthlyCharges, na.rm = TRUE),
23     monthly_mean = mean(MonthlyCharges, na.rm = TRUE),
24     monthly_median = median(MonthlyCharges, na.rm = TRUE),
25
26     total_min = min(as.numeric(TotalCharges), na.rm = TRUE),
27     total_max = max(as.numeric(TotalCharges), na.rm = TRUE),
28     total_mean = mean(as.numeric(TotalCharges), na.rm = TRUE),
29     total_median = median(as.numeric(TotalCharges), na.rm = TRUE)
30   )
31
32 print("=== Summary statistics (raw) ===")
33 print(summary_stats)
34
35 # Churn distribution (bar chart)
36 p_churn <- ggplot(telco, aes(x = Churn, fill = Churn)) +
37   geom_bar() +
38   labs(title = "Churn Distribution (Raw Data)", x = "Churn", y = "Count") +
39   theme_minimal() +
40   scale_fill_manual(values = c("No" = "#5DADE2", "Yes" = "#E74C3C"))
41
42 print(p_churn)
43
44 # Boxplots: churn vs non-churn for charges

```

```

45
46 # Boxplot for MonthlyCharges
47 p_box_monthly <- ggplot(telco, aes(x = Churn, y = MonthlyCharges, fill = Churn)) +
48   geom_boxplot(outlier.alpha = 0.2) +
49   labs(title = "Monthly Charges by Churn Status (Raw)", x = "Churn", y = "MonthlyCharges") +
50   theme_minimal() +
51   scale_fill_manual(values = c("No" = "#5DADE2", "Yes" = "#E74C3C"))
52
53 print(p_box_monthly)
54
55 # Boxplot for TotalCharges
56 # Convert TotalCharges to numeric (coerce NAs) just for plotting
57 telco$TotalCharges_num <- suppressWarnings(as.numeric(telco$TotalCharges))
58 p_box_total <- ggplot(telco, aes(x = Churn, y = TotalCharges_num, fill = Churn)) +
59   geom_boxplot(outlier.alpha = 0.2) +
60   labs(title = "Total Charges by Churn Status (Raw)", x = "Churn", y = "TotalCharges") +
61   theme_minimal() +
62   scale_fill_manual(values = c("No" = "#5DADE2", "Yes" = "#E74C3C"))
63
64 print(p_box_total)
65
66 # Correlation heatmap for numerical features
67 num_df <- telco %>%
68   select(
69     tenure,
70     MonthlyCharges,
71     TotalCharges_num,
72     # also include SeniorCitizen (which is numeric / integer)
73     SeniorCitizen
74   )
75
76 # Compute correlation matrix (use "pairwise complete observations" to handle NAs)
77 corr_mat <- cor(num_df, use = "pairwise.complete.obs")
78
79 print("=== Correlation matrix (raw) ===")
80 print(round(corr_mat, 3))
81
82 # Plot correlation heatmap with corrplot
83 corrplot(corr_mat, method = "color", type = "upper",
84   addCoef.col = "black", number.cex = 0.8,
85   tl.col = "black", tl.srt = 45,
86   title = "Correlation Heatmap (Raw Numeric Features)")
87

```

2. Preprocessing

- Handle missing data.
- Encode categorical variables (One-Hot).
- Normalize numerical features.

R Code

```
Preprocessing.R
1
2 library(dplyr)
3 library(caret)      # for dummyVars and preProcess
4 library(tidyr)      # for replacing missing values
5 # Load data
6 data_path <- "D:/Msc/datasets/3/Churn.csv" # Adjust path
7 telco <- read.csv(data_path, stringsAsFactors = FALSE)
8
9 # Handle missing data
10 telco$TotalCharges[telco$TotalCharges == ""] <- NA
11 telco$TotalCharges <- as.numeric(telco$TotalCharges)
12
13 # Remove rows with missing values
14 telco_clean <- telco %>%
15   drop_na()
16
17 # Encode categorical variables (One-Hot).
18 # Identify categorical columns (excluding target 'Churn')
19 cat_cols <- names(telco_clean)[apply(telco_clean, function(x) is.character(x))]
20 cat_cols <- setdiff(cat_cols, "Churn") # exclude target
21
22 # One-Hot encoding using caret::dummyVars
23 dummies <- dummyVars(~ ., data = telco_clean[, cat_cols])
24 cat_data <- predict(dummies, newdata = telco_clean[, cat_cols])
25 cat_data <- as.data.frame(cat_data)
26
27 # Normalize numerical features. Identify numeric columns
28 num_cols <- c("tenure", "MonthlyCharges", "TotalCharges")
29 num_data <- telco_clean[, num_cols]
30
31 # Normalize using caret::preProcess (min-max scaling)
32 preproc <- preProcess(num_data, method = c("range"))
33 num_scaled <- predict(preproc, num_data)
34
35 # Combine processed features .Keep target 'Churn' as factor
36 target <- telco_clean$Churn
37 target <- factor(target, levels = c("No", "Yes"))
38
39 # Combine numeric, encoded categorical, and target
40 telco_processed <- cbind(num_scaled, cat_data, Churn = target)
41
42 # View final preprocessed dataset
43 str(telco_processed)
44 head(telco_processed)
45
```

3. Feature Engineering & Reduction

- Create tenure groups (short, medium, long).

```
> # Check distribution
> table(telco$TenureGroup)
```

Short	Medium	Long
2175	2618	2239

- Total monthly charges per contract type.

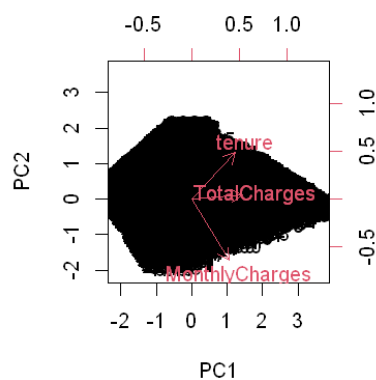
```
> print(total_monthly_charges)
# A tibble: 3 × 4
  Contract      TotalMonthlyCharges AvgMonthlyCharges Count
  <chr>          <dbl>          <dbl> <int>
1 Month-to-month    257294.         66.4   3875
2 One year          95797.         65.1   1472
3 Two year         102570.         60.9   1685
```

- Apply PCA for numerical features.

```
> # PCA summary
> summary(pca_result)
Importance of components:
```

	PC1	PC2	PC3
Standard deviation	1.4764	0.8722	0.24381
Proportion of Variance	0.7266	0.2536	0.01981
Cumulative Proportion	0.7266	0.9802	1.00000

```
>
```



R Code

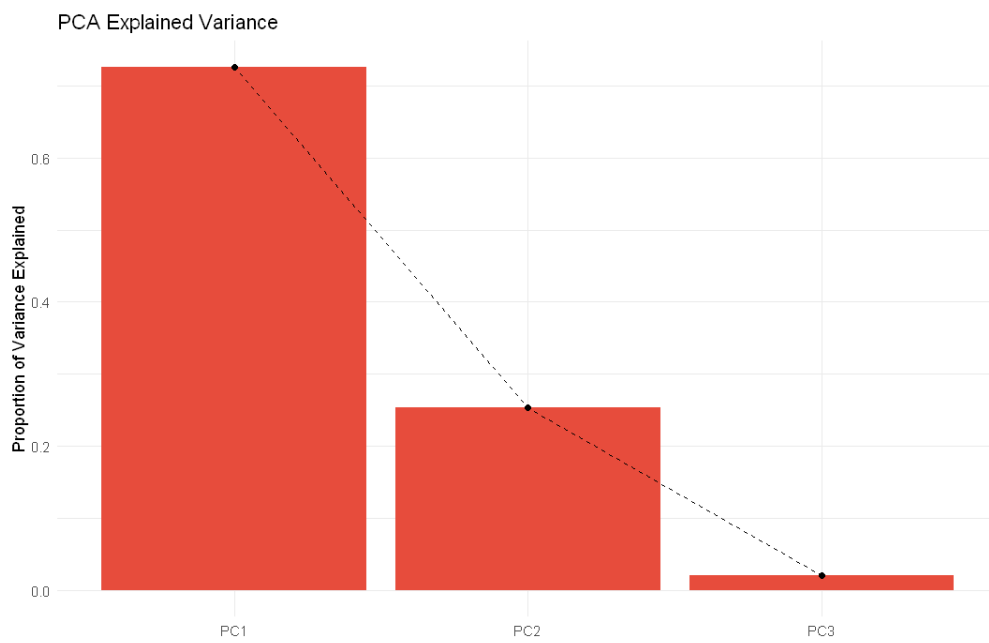
```

1 |
2 | library(dplyr)
3 | library(ggplot2)
4 | library(caret)
5 |
6 | # Assuming telco_processed is available
7 | # If starting from raw data, use preprocessing script first
8 | data_path <- "D:/Msc/datasets/3/Churn.csv"
9 | telco <- read.csv(data_path, stringsAsFactors = FALSE)
10 |
11 | # Convert TotalCharges to numeric
12 | telco$TotalCharges[telco$TotalCharges == ""] <- NA
13 | telco$TotalCharges <- as.numeric(telco$TotalCharges)
14 | telco <- telco %>% drop_na()
15 |
16 | # Create tenure groups
17 | # Short: <= 12 months, Medium: 13-48 months, Long: > 48 months
18 | telco <- telco %>%
19 |   mutate(TenureGroup = case_when(
20 |     tenure <= 12 ~ "Short",
21 |     tenure <= 48 ~ "Medium",
22 |     TRUE ~ "Long"
23 |   ))
24 |
25 | telco$TenureGroup <- factor(telco$TenureGroup, levels = c("Short", "Medium", "Long"))
26 |
27 | # Check distribution
28 | table(telco$TenureGroup)
29 |
30 | # Total monthly charges per contract type. Summarize MonthlyCharges grouped by Contract
31 | total_monthly_charges <- telco %>%
32 |   group_by(Contract) %>%
33 |   summarise(TotalMonthlyCharges = sum(MonthlyCharges, na.rm = TRUE),
34 |             AvgMonthlyCharges = mean(MonthlyCharges, na.rm = TRUE),
35 |             Count = n())
36 |
37 | print(total_monthly_charges)
38 |
39 | # Optional: visualize
40 | ggplot(total_monthly_charges, aes(x = Contract, y = TotalMonthlyCharges, fill = Contract)) +
41 |   geom_bar(stat = "identity") +
42 |   labs(title = "Total Monthly Charges per Contract Type", y = "Total Monthly Charges") +
43 |   theme_minimal() +
44 |   scale_fill_brewer(palette = "Set2")
45 |
46 | # PCA for numerical features
47 | # Select numeric features for PCA
48 | num_features <- telco %>%
49 |   select(tenure, MonthlyCharges, TotalCharges)
50 |
51 | # Standardize numeric features
52 | preproc <- preProcess(num_features, method = c("center", "scale"))
53 | num_scaled <- predict(preproc, num_features)
54 |
55 | # Apply PCA
56 | pca_result <- prcomp(num_scaled, center = FALSE, scale. = FALSE)
57 |
58 | # PCA summary
59 | summary(pca_result)
60 |
61 | # Scree plot (explained variance)
62 | explained_var <- pca_result$sdev^2 / sum(pca_result$sdev^2)
63 | plot(explained_var, type = "b", xlab = "Principal Component", ylab = "Proportion of Variance Explained",
64 |      main = "PCA Scree Plot")
65 |
66 | # Optional: biplot
67 | biplot(pca_result, scale = 0)
68 |

```

4. Second EDA (Post-Processing)

- Feature importance visualization (Random Forest).
- PCA explained variance chart.



Code

```
EDA_2.R
Source on Save
Run

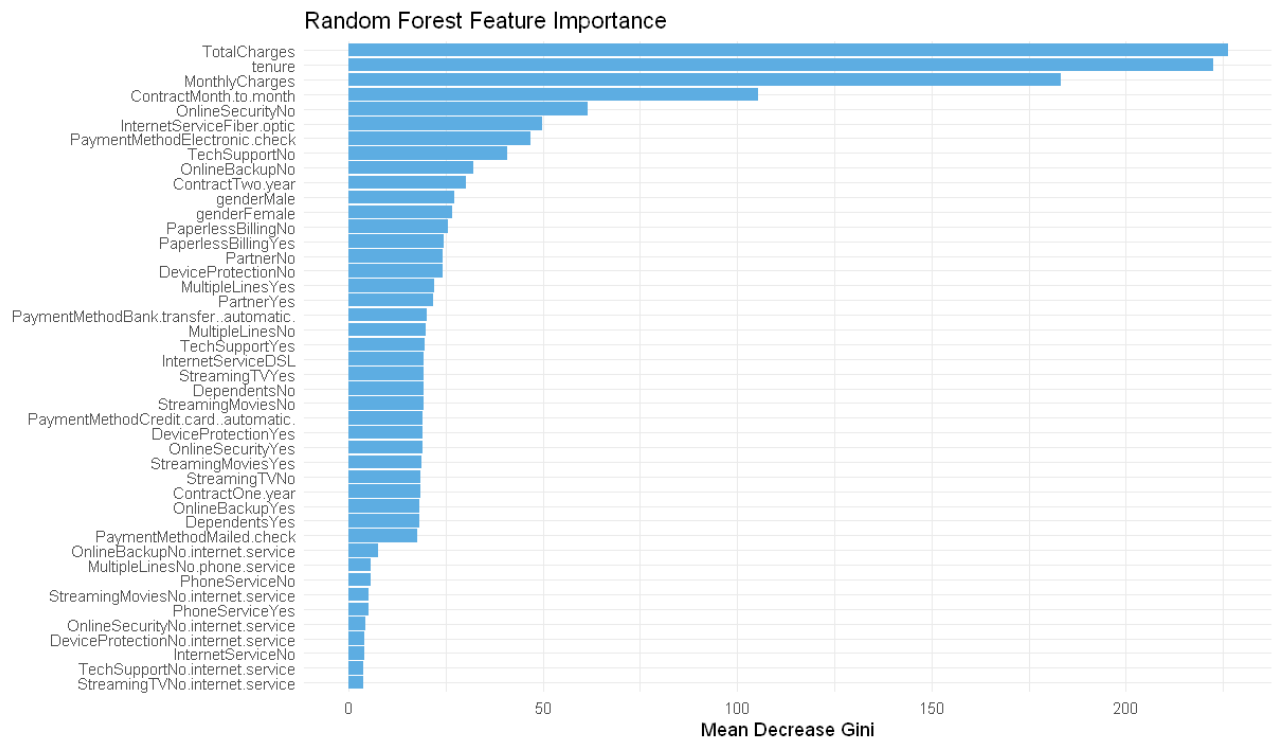
1 library(dplyr)
2 library(randomForest)
3 library(ggplot2)
4 library(caret)
5 library(tidyr)
6
7 # -----
8 # Load and preprocess data
9 # -----
10 data_path <- "D:/Msc/datasets/3/Churn.csv"
11 telco <- read.csv(data_path, stringsAsFactors = FALSE)
12
13 # Remove customerID
14 telco <- telco %>% select(-customerID)
15
16 # Convert TotalCharges to numeric and drop missing values
17 telco$TotalCharges[telco$TotalCharges == ""] <- NA
18 telco$TotalCharges <- as.numeric(telco$TotalCharges)
19 telco <- telco %>% drop_na()
20
21 # -----
22 # Create tenure groups
23 # -----
24 telco <- telco %>%
25   mutate(TenureGroup = case_when(
26     tenure <= 12 ~ "Short",
27     tenure <= 48 ~ "Medium",
28     TRUE ~ "Long"
29   ))
30 telco$TenureGroup <- factor(telco$TenureGroup, levels = c("Short", "Medium", "Long"))
31
32 # -----
33 # One-Hot encode categorical variables
34 # -----
35 cat_cols <- names(telco)[sapply(telco, is.character)]
36 cat_cols <- setdiff(cat_cols, "Churn") # exclude target
37 dummies <- dummyVars(~ ., data = telco[, cat_cols])
38 cat_data <- predict(dummies, newdata = telco[, cat_cols])
39 cat_data <- as.data.frame(cat_data)
40
41 # Fix column names
42 names(cat_data) <- make.names(names(cat_data))
43
44 # Combine with numeric features
45 telco_final <- cbind(num_scaled, cat_data, Churn = target)
```

```

46
47 # Train Random Forest
48 set.seed(123)
49 rf_model <- randomForest(Churn ~ ., data = telco_final, importance = TRUE, ntree = 100)
50
51 # -----
52 # Normalize numeric features
53 # -----
54 num_cols <- c("tenure", "MonthlyCharges", "TotalCharges")
55 preproc <- preProcess(telco[, num_cols], method = c("range"))
56 num_scaled <- predict(preproc, telco[, num_cols])
57
58 # -----
59 # Combine processed features + target
60 # -----
61 target <- factor(telco$Churn, levels = c("No", "Yes"))
62 telco_final <- cbind(num_scaled, cat_data, Churn = target)
63
64 # -----
65 # Feature Importance using Random Forest
66 # -----
67 set.seed(123)
68 rf_model <- randomForest(Churn ~ ., data = telco_final, importance = TRUE, ntree = 100)
69
70 importance_df <- as.data.frame(importance(rf_model))
71 importance_df$Feature <- rownames(importance_df)
72 importance_df <- importance_df %>% arrange(desc(MeanDecreaseGini))
73
74 ggplot(importance_df, aes(x = reorder(Feature, MeanDecreaseGini), y = MeanDecreaseGini)) +
75   geom_bar(stat = "identity", fill = "#5DADE2") +
76   coord_flip() +
77   labs(title = "Random Forest Feature Importance", y = "Mean Decrease Gini", x = "") +
78   theme_minimal()
79
80 # -----
81 # PCA explained variance chart
82 # -----
83 pca_result <- prcomp(num_scaled, center = TRUE, scale. = TRUE)
84 explained_var <- pca_result$sdev^2 / sum(pca_result$sdev^2)
85 explained_df <- data.frame(
86   PC = paste0("PC", 1:length(explained_var)),
87   Variance = explained_var
88 )
89
90 ggplot(explained_df, aes(x = PC, y = Variance)) +
91   geom_bar(stat = "identity", fill = "#E74C3C") +
92   geom_line(aes(group = 1), color = "black", linetype = "dashed") +
93   geom_point(color = "black") +
94   labs(title = "PCA Explained Variance", y = "Proportion of Variance Explained", x = "") +
95   theme_minimal()
96

```

5. ML Modeling: Train Decision Tree, Random Forest, Gradient Boosting (XGBoost).



Code

```

1
2 library(dplyr)
3 library(caret)
4 library(randomForest)
5 library(rpart)      # Decision Tree
6 library(xgboost)    # XGBoost
7 library(pROC)       # ROC-AUC
8
9 #Load and preprocess data
10 data_path <- "D:/Msc/datasets/3/Churn.csv"
11 telco <- read.csv(data_path, stringsAsFactors = FALSE)
12
13 # Remove customerID
14 telco <- telco %>% select(-customerID)
15
16 # Convert TotalCharges to numeric and remove NAs
17 telco$TotalCharges[telco$TotalCharges == ""] <- NA
18 telco$TotalCharges <- as.numeric(telco$TotalCharges)
19 telco <- telco[complete.cases(telco), ]
20
21 # Create tenure groups
22 telco <- telco %>%
23   mutate(TenureGroup = case_when(
24     tenure <= 12 ~ "Short",
25     tenure <= 48 ~ "Medium",
26     TRUE ~ "Long"
27   ))
28 telco$TenureGroup <- factor(telco$TenureGroup, levels = c("Short", "Medium", "Long"))
29
30 # Encode categorical variables (One-Hot)
31 cat_cols <- names(telco)[sapply(telco, is.character)]
32 cat_cols <- setdiff(cat_cols, "Churn") # exclude target
33 dummies <- dummyVars(~ ., data = telco[, cat_cols])
34 cat_data <- predict(dummies, newdata = telco[, cat_cols])
35 cat_data <- as.data.frame(cat_data)
36 names(cat_data) <- make.names(names(cat_data)) # clean column names
37
38 # Normalize numeric features
39 num_cols <- c("tenure", "MonthlyCharges", "TotalCharges")
40 preproc <- preProcess(telco[, num_cols], method = c("range"))
41 num_scaled <- predict(preproc, telco[, num_cols])
42
43 # Combine numeric + categorical + target
44 target <- factor(telco$Churn, levels = c("No", "Yes"))
45 telco_final <- cbind(num_scaled, cat_data, Churn = target)

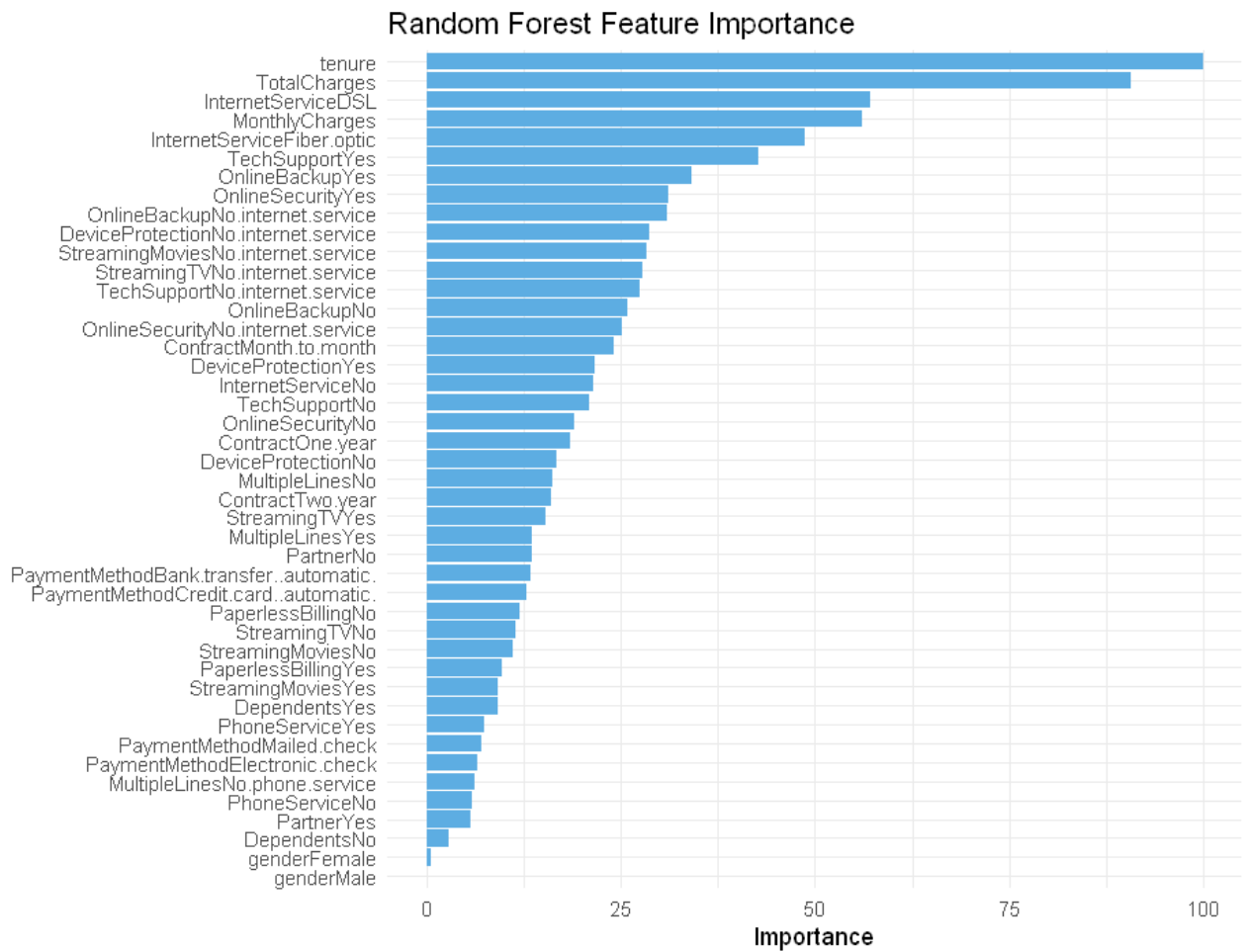
```

```

46
47 #Train/Test Split
48 set.seed(123)
49 train_index <- createDataPartition(telco_final$Churn, p = 0.7, list = FALSE)
50 train_data <- telco_final[train_index, ]
51 test_data <- telco_final[-train_index, ]
52
53 #Decision Tree
54 dt_model <- rpart(Churn ~ ., data = train_data, method = "class")
55 dt_pred <- predict(dt_model, test_data, type = "class")
56
57 # Confusion matrix and accuracy
58 confusionMatrix(dt_pred, test_data$Churn)
59
60 #Random Forest
61 rf_model <- randomForest(Churn ~ ., data = train_data, ntree = 100, importance = TRUE)
62 rf_pred <- predict(rf_model, test_data)
63
64 # Confusion matrix and accuracy
65 confusionMatrix(rf_pred, test_data$Churn)
66
67 # Feature importance
68 importance_df <- as.data.frame(importance(rf_model))
69 importance_df$Feature <- rownames(importance_df)
70 importance_df <- importance_df %>% arrange(desc(MeanDecreaseGini))
71
72 ggplot(importance_df, aes(x = reorder(Feature, MeanDecreaseGini), y = MeanDecreaseGini)) +
73   geom_bar(stat = "identity", fill = "#5DADE2") +
74   coord_flip() +
75   labs(title = "Random Forest Feature Importance", y = "Mean Decrease Gini", x = "") +
76   theme_minimal()
77
78 # XGBoost. # XGBoost requires numeric matrix and numeric labels (0/1)
79 train_matrix <- as.matrix(train_data %>% select(-Churn))
80 test_matrix <- as.matrix(test_data %>% select(-Churn))
81 train_label <- ifelse(train_data$Churn == "Yes", 1, 0)
82 test_label <- ifelse(test_data$Churn == "Yes", 1, 0)
83
84 xgb_model <- xgboost(data = train_matrix, label = train_label,
85                     nrounds = 100, objective = "binary:logistic", verbose = 0)
86
87 xgb_pred_prob <- predict(xgb_model, test_matrix)
88 xgb_pred <- ifelse(xgb_pred_prob > 0.5, "Yes", "No")
89 xgb_pred <- factor(xgb_pred, levels = c("No", "Yes"))
90
91 # Confusion matrix and accuracy
92 confusionMatrix(xgb_pred, test_data$Churn)
93
94 #ROC-AUC for all models
95
96 # Decision Tree
97 dt_prob <- predict(dt_model, test_data)[,2]
98 roc_dt <- roc(as.numeric(test_data$Churn)-1, dt_prob)
99 print(paste("Decision Tree AUC:", auc(roc_dt)))
100
101 # Random Forest
102 rf_prob <- predict(rf_model, test_data, type = "prob")[,2]
103 roc_rf <- roc(as.numeric(test_data$Churn)-1, rf_prob)
104 print(paste("Random Forest AUC:", auc(roc_rf)))
105
106 # XGBoost
107 roc_xgb <- roc(test_label, xgb_pred_prob)
108 print(paste("XGBoost AUC:", auc(roc_xgb)))
109

```

6. Evaluation: ROC-AUC, accuracy, recall, Stratified k-fold CV



R Code

```

1
2 library(dplyr)
3 library(caret)
4 library(randomForest)
5 library(rpart)
6 library(xgboost)
7 library(pROC)
8 library(ggplot2)
9
10 data_path <- "D:/Msc/datasets/3/Churn.csv"
11 telco <- read.csv(data_path, stringsAsFactors = FALSE)
12
13 # Remove customerID
14 telco <- telco %>% select(-customerID)
15
16 # Convert TotalCharges to numeric and drop missing values
17 telco$TotalCharges[telco$TotalCharges == ""] <- NA
18 telco$TotalCharges <- as.numeric(telco$TotalCharges)
19 telco <- telco[complete.cases(telco), ]
20
21 # Create tenure groups
22 telco <- telco %>%
23   mutate(TenureGroup = case_when(
24     tenure <= 12 ~ "Short",
25     tenure <= 48 ~ "Medium",
26     TRUE ~ "Long"
27   ))
28 telco$TenureGroup <- factor(telco$TenureGroup, levels = c("Short", "Medium", "Long"))
29
30 # One-hot encode categorical variables
31 cat_cols <- names(telco)[sapply(telco, is.character)]
32 cat_cols <- setdiff(cat_cols, "Churn")
33 dummies <- dummyVars(~ ., data = telco[, cat_cols])
34 cat_data <- predict(dummies, newdata = telco[, cat_cols])
35 cat_data <- as.data.frame(cat_data)
36 names(cat_data) <- make.names(names(cat_data)) # clean column names
37
38 # Normalize numeric features
39 num_cols <- c("tenure", "MonthlyCharges", "TotalCharges")
40 preproc <- preProcess(telco[, num_cols], method = c("range"))
41 num_scaled <- predict(preproc, telco[, num_cols])
42

```



```

43 # Combine features + target
44 target <- factor(telco$Churn, levels = c("No", "Yes"))
45 telco_final <- cbind(num_scaled, cat_data, Churn = target)
46
47 #Define train Control for Stratified k-Fold CV
48
49 set.seed(123)
50 cv_ctrl <- trainControl(
51   method = "cv",
52   number = 5,                # 5-fold CV
53   classProbs = TRUE,        # needed for ROC
54   summaryFunction = twoClassSummary,
55   savePredictions = TRUE
56 )
57
58
59 #Decision Tree with CV
60
61 set.seed(123)
62 dt_model <- train(
63   Churn ~ .,
64   data = telco_final,
65   method = "rpart",
66   trControl = cv_ctrl,
67   metric = "ROC" # optimize for ROC-AUC
68 )
69
70 # CV metrics
71 dt_model$results
72
73 # Random Forest with CV
74
75 set.seed(123)
76 rf_model <- train(
77   Churn ~ .,
78   data = telco_final,
79   method = "rf",
80   trControl = cv_ctrl,
81   metric = "ROC",
82   importance = TRUE
83 )
84

```

```

85 rf_model$results
86
87 # Feature importance plot
88 #importance_df <- as.data.frame(varImp(rf_model)$importance)
89 #importance_df$Feature <- rownames(importance_df)
90 #importance_df <- importance_df %>% arrange(desc(Overall))
91
92 #ggplot(importance_df, aes(x = reorder(Feature, Overall), y = Overall)) +
93 #  geom_bar(stat = "identity", fill = "#5DADE2") +
94 #  coord_flip() +
95 #  labs(title = "Random Forest Feature Importance (CV)", y = "Importance", x = "") +
96 #  theme_minimal()
97 importance_df <- as.data.frame(varImp(rf_model)$importance)
98 importance_df$Feature <- rownames(importance_df)
99
100 # Find the first numeric column automatically
101 importance_col <- names(importance_df)[sapply(importance_df, is.numeric)][1]
102
103 importance_df <- importance_df %>%
104   arrange(desc(!sym(importance_col)))
105
106 # Plot
107 ggplot(importance_df, aes(x = reorder(Feature, !sym(importance_col)), y = !sym(importance_col))) +
108   geom_bar(stat = "identity", fill = "#5DADE2") +
109   coord_flip() +
110   labs(title = "Random Forest Feature Importance", y = "Importance", x = "") +
111   theme_minimal()
112
113
114 # XGBoost with CV
115
116 set.seed(123)
117 xgb_model <- train(
118   Churn ~ .,
119   data = telco_final,
120   method = "xgbTree",
121   trControl = cv_ctrl,
122   metric = "ROC"
123 )
124
125 xgb_model$results
126
127
128 # Evaluate on full dataset (optional)
129
130 # Predictions for ROC, Accuracy, Recall
131 models <- list(DecisionTree = dt_model, RandomForest = rf_model, XGBoost = xgb_model)
132
133 for (name in names(models)) {
134   pred_prob <- predict(models[[name]], telco_final, type = "prob")[, "Yes"]
135   pred_class <- predict(models[[name]], telco_final)
136
137   cm <- confusionMatrix(pred_class, telco_final$Churn)
138   roc_obj <- roc(as.numeric(telco_final$Churn)-1, pred_prob)
139
140   cat("\n===== \n")
141   cat(name, "\n")
142   print(cm)
143   cat("ROC-AUC:", auc(roc_obj), "\n")
144 }
145

```

36/64 (Top Level) R Script

7. Interpretation

- Identify top churn drivers.

The top churn drivers are

#	Feature	Interpretation
1	Contract (Month-to-Month vs Long-term)	Customers on month-to-month contracts are more likely to churn.
2	Tenure / TenureGroup	Customers with short tenure are more likely to leave.

- Suggest retention strategies.
 - a) Focus on early tenure engagement
 - b) Contract incentives
 - c) Personalized service offerings
 - d) Monitoring high-risk customer segments.